

FounderLink Microservices End-to-End Workflows

Generated: May 4, 2026

This document explains how every major user action moves from the Angular frontend, through the API Gateway, into each Spring Boot microservice, and finally into storage, service-to-service calls, RabbitMQ notifications, or email delivery.

System Overview

FounderLink is a full-stack microservices platform for founders, investors, co-founders, and admins. The Angular frontend uses a central AppStore to manage forms, session state, service-page actions, and HTTP calls. The frontend talks to the backend through one gateway URL: `http://localhost:8080`.

Browser user -> Angular frontend -> API Gateway -> Microservice controller -> Service layer
-> Repository / Feign / RabbitMQ -> Response -> Angular state update

Shared Runtime Pattern

- User performs an action in the Angular UI.
- Angular calls a method in AppStore, such as `login()`, `saveStartup()`, `createInvestment()`, `sendInvite()`, or `sendMessage()`.
- HttpClient sends the request to `http://localhost:8080`.
- API Gateway routes the request by path.
- Protected services validate the JWT from the Authorization header.
- The controller calls the service layer.
- The service layer saves data in PostgreSQL, reads Redis cache where used, calls other services through Feign, or publishes notification events to RabbitMQ.
- The frontend receives the response and updates local Angular signals/state.

Gateway Routes

- `/auth/**` -> auth-service -> port 8081
- `/users/**` -> user-service -> port 8082
- `/startups/**` -> startup-service -> port 8083
- `/investments/**` -> investment-service -> port 8084
- `/teams/**` -> team-service -> port 8085
- `/messages/**` -> messaging-service -> port 8086
- `/notifications/**` and `/api/notifications/**` -> notification-service -> port 8087

API Gateway Workflow

The gateway is the single backend entry point for the frontend. It keeps the Angular app independent from individual service URLs and centralizes CORS and Swagger aggregation.

- Angular builds a request URL using `API_BASE`, usually `http://localhost:8080`.
- The browser sends the request to the gateway.
- Gateway matches the request path, for example `/startups`.
- Gateway forwards the request to the configured service URL, for example `startup-service:8083`.
- The target service returns JSON or text to the gateway.
- Gateway returns the response to Angular.

Auth Service Workflow

Auth service owns registration, OTP verification, login, role assignment, JWT generation, and password reset. It also syncs profile data into user-service.

Frontend Actions and Endpoints

- Register -> POST /auth/register -> create pending user, assign role, sync profile, send OTP.
- Verify OTP -> POST /auth/verify-otp -> enable account after OTP validation.
- Resend OTP -> POST /auth/resend-otp -> generate and send a new OTP.
- Login -> POST /auth/login -> validate credentials and return JWT.
- Forgot password -> POST /auth/forgot-password -> generate password reset OTP.
- Verify reset OTP -> POST /auth/forgot-password/verify -> validate reset OTP before password change.
- Reset password -> POST /auth/reset-password -> update encoded password.

Registration Flow

- User fills role, personal info, and profile details on the Angular auth page.
- AppStore.register() posts email, password, role, name, and bio to /auth/register.
- AuthController passes the request to AuthService.register().
- Auth service creates or refreshes a pending user record in PostgreSQL.
- Password is encoded with Spring Security.
- Requested role is normalized and linked through the user-role table.
- Auth service calls user-service through Feign at /users/internal/sync.
- Auth service publishes an OTP notification event to RabbitMQ.
- Frontend stores pending verification email and shows the OTP step.

Login Flow

- User submits email, password, and selected role.
- Auth service finds the user by email and checks the account is enabled.
- If not verified, a new OTP is generated and sent, and login fails with a helpful message.
- Password is checked with PasswordEncoder.matches().
- Selected role must match assigned role.
- Auth service ensures the profile exists in user-service.
- JWT is generated with user id as subject and roles as claims.
- Angular stores sessionId and sessionToken, then refreshes backend data.

User Service Workflow

User service owns public profile information and user directory summaries. Authentication data stays in auth-service; profile data stays here.

Endpoints

- POST /users -> create profile directly.
- GET /users -> return full profile list.
- GET /users/directory -> return profile summaries for frontend lists.
- GET /users/{id} -> return one profile.
- GET /users/internal/{id} -> service-to-service user lookup.
- POST /users/internal/sync -> auth service syncs profile after registration/login recovery.
- PUT /users/{id} -> save profile edits from frontend.

Save Profile Flow

- User edits profile fields on the Profile Service page.
- `AppStore.saveProfile()` sends `PUT /users/{currentUserId}` with JWT.
- User service validates the JWT and request body.
- `UserService.update()` checks email uniqueness.
- Profile fields are copied into the existing entity.
- PostgreSQL is updated.
- Redis cache is refreshed through `@CachePut`.
- Angular merges the returned profile into local state.

Internal Lookup Flow

Startup, investment, team, and messaging services call user-service internally to get names and emails. This avoids copying profile data into every service.

Startup Service Workflow

Startup service owns startup listings, founder ownership, admin review status, and startup lifecycle notifications.

Endpoints

- `GET /startups` -> return listings for dashboards, discovery, and admin.
- `POST /startups` -> founder creates a listing.
- `GET /startups/{id}` -> used by other services, especially investment-service.
- `PUT /startups/{id}` -> founder edits listing or admin updates status.
- `DELETE /startups/{id}` -> admin deletes a listing.

Create Startup Flow

- Founder opens Startup Service page and fills startup name, industry, stage, funding goal, problem, solution, and pitch.
- `AppStore.saveStartup()` sends `POST /startups`.
- Gateway forwards request to startup-service.
- JWT filter identifies the logged-in founder.
- `StartupService.create()` clears any sent id and sets `userId` from the authenticated token.
- Startup is saved in PostgreSQL with initial status.
- Startup service gets founder email through user-service Feign client.
- Startup creation notification is published to RabbitMQ.
- Angular receives saved startup and adds it to local startup state.

Admin Review Flow

- Admin opens Admin Service page and chooses approve or reject.
- Angular calls `reviewStartup()`, which sends `PUT /startups/{id}`.
- Startup service updates the status.
- If status changes to accepted or rejected, startup service notifies the founder through RabbitMQ.
- Approved startups become visible on the Discovery page.

Investment Service Workflow

Investment service owns investment requests and investment status transitions. It uses startup-service and user-service to notify founders about new investment interest.

Endpoints

- `POST /investments` -> investor creates pending investment request.
- `GET /investments` -> frontend refresh path for current containers.

- GET /investments/startup/{id} -> founder reviews requests for their startup.
- GET /investments/investor/{id} -> investor portfolio view.
- PUT /investments/{id}/status -> founder approves, rejects, or completes investment.

Create Investment Flow

- Investor selects an approved startup and enters an amount.
- Angular sends POST /investments with startup id and amount.
- Investment service validates JWT and sets investorId from the authenticated user.
- Status is set to PENDING.
- Investment is saved in PostgreSQL.
- Investment service calls startup-service to load the startup and founder id.
- Investment service calls user-service to load founder email.
- Notification event is published to RabbitMQ for the founder.
- Angular adds the investment to the investor portfolio and updates progress calculations.

Founder Status Update Flow

- Founder opens Investment Service page and sees requests for their startups.
- Founder can approve or reject a pending request.
- Approved requests can later be marked completed.
- Angular sends PUT /investments/{id}/status.
- Investment service updates status in PostgreSQL.
- Frontend updates the cards and status hints.

Team Service Workflow

Team service owns invitations and accepted startup memberships.

Endpoints

- POST /teams/invite -> founder/admin invites a cofounder.
- POST /teams/join -> cofounder accepts their invitation.
- GET /teams/startup/{id} -> show members/invites for a startup.
- GET /teams/my -> show active teams for logged-in user.

Invite Cofounder Flow

- Founder selects startup, invitee, and role.
- AppStore.sendInvite() sends POST /teams/invite.
- Team service creates or updates a team row for that startup and user.
- Status is set to PENDING.
- Team service calls user-service to find the invited user email.
- Notification event is published to RabbitMQ.
- Angular adds the invite to local state.

Accept Invitation Flow

- Cofounder opens Team Service page and sees incoming invites.
- Angular sends POST /teams/join.
- Team service checks the JWT user matches the invited user unless the requester is admin.
- Existing pending invitation is loaded.
- Status changes from PENDING to ACTIVE.
- Notification event confirms the join.
- Frontend maps ACTIVE to accepted invite state.

Messaging Service Workflow

Messaging service owns conversations and messages. It also ensures users can access only their own conversations.

Endpoints

- POST /messages/conversation -> create or reuse a conversation between two users.
- POST /messages -> store a message and notify recipient.
- GET /messages/conversation/{id} -> read conversation thread.
- GET /messages/user/{id}/conversations -> list conversations for logged-in user.

Open Conversation Flow

- User selects a message contact from the service page.
- Angular calls AppStore.openConversation(partnerId).
- Frontend sends POST /messages/conversation with current user and partner ids.
- Messaging service confirms the authenticated user is one participant.
- Existing conversation is reused if found in either participant order.
- If none exists, a new conversation is saved in PostgreSQL.
- Angular sets the selected conversation id.

Send Message Flow

- User types a message and clicks send.
- Angular sends POST /messages with conversation id, sender id, and content.
- Messaging service checks sender id matches JWT subject.
- Conversation is loaded and recipient is resolved.
- Message is saved with current timestamp.
- Service calls user-service for sender and recipient summary data.
- RabbitMQ notification is published for recipient email.
- Frontend appends the message locally and also refreshes messages periodically every 5 seconds.

Notification Service Workflow

Notification service decouples email delivery from user-facing operations. Producers only publish events; this service consumes those events and sends email.

Endpoint and Event

- GET /api/notifications/status -> frontend checks notification-service availability.
- notification.queue -> RabbitMQ listener receives notification events.

Email Event Processing

- Auth, startup, investment, team, or messaging service creates a NotificationEvent.
- Producer sends the event to notification.exchange with routing key notification.routing-key.
- RabbitMQ places the message on notification.queue.
- NotificationConsumer receives the event with @RabbitListener.
- NotificationService.sendEmail() validates SMTP username and password.
- Spring Mail sends an email with subject FounderLink Notification.
- If delivery fails, the consumer rejects without requeueing.

Config Server Workflow

Config server provides centralized service configuration. It is not called by the frontend. Backend services use it during startup to load configuration such as ports, service URLs, database settings, RabbitMQ settings, and discovery settings.

- Config server starts on port 8888.
- Each service starts and reads its configuration from config server or local fallback properties.
- Service-specific files live under config-server/src/main/resources/config.

- Centralized config reduces duplication across microservices.

Eureka Server Workflow

Eureka server provides service discovery. It is also not called by the frontend.

- Eureka server starts on port 8761.
- Microservices register themselves with Eureka during startup.
- Gateway and Feign clients can discover services through the registry.
- This helps when services restart, move, or scale.

Complete Example: Investor Creates Investment

Investor clicks Invest -> Angular AppStore.createInvestment() -> POST /investments -> API Gateway -> investment-service -> JWT validation -> PostgreSQL save -> Feign startup-service -> Feign user-service -> RabbitMQ -> notification-service -> founder email -> Angular state update

- Investor finds an approved startup on Discovery page.
- Investor clicks Invest and enters amount.
- Angular sends the request through the gateway with JWT.
- Investment service saves a pending request.
- Investment service finds the startup founder using startup-service.
- Investment service finds founder email using user-service.
- Notification event is published to RabbitMQ.
- Notification service sends email to founder.
- Frontend updates portfolio and dashboard progress.

Summary

The application follows a consistent microservice workflow: Angular action, gateway route, JWT validation, controller, service layer, repository persistence, optional Feign lookup, optional RabbitMQ event, and frontend state refresh. Each service owns one business area, which keeps authentication, profiles, startups, investments, teams, messages, and email delivery separated but connected.